NAGP 2026 ASSIGNMENT

Task Planner Multi-Tier Application

Enterprise Kubernetes Orchestration & Continuous Deployment Blueprint

Developer	Sidhant Gupta
Email	guptasidhant1997@gmail.com
Contact	+91-9996764596
Repository	github.com/sidhant97/todo-app-docker-kubernetes
Date	June 20, 2026

1. Architectural Overview & Requirements

This engineering blueprint provides a holistic framework for the deployment of a highly available, fault-tolerant, multi-tier **Todo Task Planner Application** natively orchestrated within Google Kubernetes Engine (GKE). The framework separates structural state from stateless execution threads to maximize compute efficiency and ensure consistent self-healing cycles.

1.1 Core Tier Topologies

- **Frontend Client Tier:** Engineered utilizing Angular frameworks. To streamline container layer complexity and network transport overheads, compiled production client distributions are embedded statically into the backend asset resolution pathways.
- **Backend Microservice Tier:** Executed as an optimized Java Spring Boot application exposing restful APIs securely inside the private cluster network overlay over dedicated execution port `8081`.
- **Stateful Database Tier:** Relational configurations are supported by a transactional MySQL engine. This layer relies explicitly on a Kubernetes `StatefulSet` abstraction to enforce strictly stable network identification hooks and reliable volume mapping.
- **Routing and Ingress Layer:** Edge traffic distribution rules are handled by the official NGINX Ingress Controller ecosystem, facilitating host-based wildcard evaluations and clean endpoint forwarding.

1.2 Resource Injections & De-coupling

Decoupled systems architecture rules are applied throughout the platform configurations. Production properties, environment profile toggles, and base parameters are encapsulated within isolated `ConfigMaps`. Sensitive variables, structural system credentials, and target database handshakes are stored inside cluster-encrypted Kubernetes `Secrets`, separating target environments from raw application source code.

2. Solution Rationalization & Design Assertions

Every operational resource component deployed inside this framework has been specified according to strict enterprise availability metrics and structural execution predictability.

2.1 Component Justification Matrix

Resource Construct	Deployment Layer	Architectural Justification & Functional Properties
3-Node GKE Compute	Infrastructure	Guarantees distributed scheduling capabilities across variable regional compute instances, enforcing zone failure isolation and elastic compute resilience.
Kubernetes Deployment	Spring Boot Backend	Manages a consistent 4-replica stateless execution pool via automated `ReplicaSet` controllers. Enables rolling revisions and active update rollbacks.
Kubernetes StatefulSet	MySQL Transaction Engine	Enforces monotonic ordinal naming structures (`mysql-0`), ensuring steady persistence mappings and unvarying network storage mounts during pod lifecycle restarts.
Dynamic PVC (1Gi)	Persistent Storage	Interacts with native Cloud Storage Drivers under `standard-rwo` profiles to claim decoupled, persistent block tracking space for live database engines.
Run-to-Completion Job	Database Initialization	Decouples transactional schema migrations from backend bootstrapping, testing health indicators via programmatic hooks prior to routing core requests.

2.2 Operational Assumptions

- **Dynamic Cloud Provisioners:** The target Google Kubernetes Engine environment is assumed to possess a fully initialized, working `standard-rwo` StorageClass.
- **DNS Loopbacks:** Network edges are configured to interact seamlessly with public wildcard mapping networks using the standard `nip.io` domain pattern linked directly to the load balancer public interface.

3. Advanced Containerization Mechanics

To reduce container layer bloat and secure execution surfaces, the application artifact compilation lifecycle uses a strict **3-Stage Multi-Stage Docker Build Process**. This pattern eliminates downstream dependencies like Node packages and compiling JDK components from the final executable environment.

3.1 Docker Construction Blueprint

```
# ============================================================
# STAGE 1: Compilation and Optimization of Angular Assets
# ============================================================
FROM node:20-alpine AS angular-builder
WORKDIR /app
COPY todo-ui/package*.json ./todo-ui/
WORKDIR /app/todo-ui
RUN npm install
COPY todo-ui .
RUN npm run build

# ============================================================
# STAGE 2: Java Compilation & Integration of Frontend Static Assets
# ============================================================
FROM eclipse-temurin:17-jdk-jammy AS java-builder
WORKDIR /app
COPY build.gradle settings.gradle gradlew ./
COPY gradle ./gradle
RUN chmod +x ./gradlew
RUN ./gradlew dependencies
COPY src ./src
COPY --from=angular-builder /app/todo-ui/dist/todo-ui/browser/ ./src/main/resources/static/
RUN ./gradlew bootJar -x copyAngularAssets

# ============================================================
# STAGE 3: Hardened Production Runtime Creation
# ============================================================
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --from=java-builder /app/build/libs/*.jar app.jar
EXPOSE 8081
USER 10001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4. Step-by-Step Deployment Protocol

To ensure proper resource resolution and prevent network handshake timeouts between tiers, manifests must be applied sequentially according to the following runbook phases.

Phase 1: Cluster Provisioning & Local Authentication

Establish the required infrastructure nodes inside GCP and bind administrative execution credentials locally:

```
# Establish a highly available 3-node GKE compute cluster
gcloud container clusters create todo-cluster     --zone us-central1-a     --num-nodes=3

# Bind execution contexts to local kubectl configurations
gcloud container clusters get-credentials todo-cluster --zone us-central1-a
```

Phase 2: Configuration Injection & Database Setup

Inject configurations and sensitive parameters before initializing the stateful components:

```
kubectl apply -f todo-secret.yaml
kubectl apply -f todo-configmap.yaml
kubectl apply -f mysql-statefulset.yaml
```

Monitor configuration progression states until the engine pod achieves stability:

```
kubectl get pods -w -l app=mysql
```

> **Database Synchronization Hook:** Once the `mysql-0` execution instance changes state to 'Running', execute the automated setup schema migration steps safely.

```
kubectl apply -f mysql-init-job.yaml
kubectl logs job/mysql-init-job
```

Phase 3: Deploying Backend Components

Apply stateless infrastructure specifications into active cluster contexts:

```
kubectl apply -f todo-deployment.yaml
kubectl get pods -l app=todo
kubectl describe svc todo-service
```

Phase 4: Setting Up Edge Routes

```
kubectl create namespace ingress-nginx
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.1/
deploy/static/provider/cloud/deploy.yaml
kubectl get svc -n ingress-nginx ingress-nginx-controller
```

> **Action Required:** Extract the public `EXTERNAL-IP` returned by the step above. Modify the host reference inside the `todo-ingress.yaml` manifest using the pattern `<EXTERNAL-IP>.nip.io` prior to running the finalize command.

```
kubectl apply -f todo-ingress.yaml
kubectl get ingress todo-ingress
```

5. Platform Resilience & Verification Runbook

The operational resilience properties of the environment can be validated manually using the standard scenarios detailed below.

5.1 Scenario A: Stateless Self-Healing Computations

This workflow simulates unexpected runtime node failures or sudden process crashes within the stateless tier.

```
# Identify running pods managed by the backend controllers
kubectl get pods -l app=todo

# Force delete an arbitrary application container pod instance
kubectl delete pod todo-deployment-856c8cdfb4-2nqj2 --grace-period=0

# Observe replication mechanisms provision a new node automatically
kubectl get pods -l app=todo -w
```

Expected Output Observation: The underlying `ReplicaSet` controller catches the state delta from the target minimum of 4 instances immediately, creating a fresh container node within seconds to restore system availability.

5.2 Scenario B: Zero-Downtime Controlled Updates

This procedure transitions live user traffic between software application versions without interrupting active execution flows.

```
# Instruct the cluster to roll out an updated container image version
kubectl set image deployment todo-deployment todo=sidhant97/todo:v2

# Monitor the rolling update implementation process
kubectl rollout status deployment todo-deployment
```

Expected Output Observation: The system executes a rolling update deployment strategy, provisioning updated v2 container profiles sequentially while keeping older v1 containers online until the new nodes pass initialization health checks.

5.3 Scenario C: Database Data Persistence Testing

This scenario tests structural persistence capabilities during a total storage driver crash or unannounced server failover events.

```
kubectl delete pod mysql-0
kubectl get pods -l app=mysql -w
```

Expected Output Observation: The `StatefulSet` infrastructure tracks the loss of the node and schedules a new instance named `mysql-0`. This new instance automatically reattaches the cloud block storage managed by the `PersistentVolumeClaim`, retaining all historical items intact.

6. Verified Core REST API Documentation

The endpoints below are exposed via the ingress routing tables for integration testing or frontend application requests.

Method	API Target Endpoint Path	Functional Operation & Structural Result
GET	`/api/todos`	Queries the backend data store to return a complete JSON collection of all managed user items.
GET	`/api/todos?completed=false`	Returns a filtered JSON payload containing only active, uncompleted task records.
GET	`/api/todos?completed=true`	Returns a filtered JSON payload containing tasks marked completed.
PUT	`/api/todos/todo/add`	Accepts a raw JSON body payload to append a validated new task record into the database.
GET	`/api/todos/todo/{id}/finish`	Updates the completion tracking flag of the task matching the specified structural path ID.
DELETE	`/api/todos/todo/{id}/delete`	Removes a specified data entry row cleanly out of the tracking tables permanently.

Sample Integration Request Execution

```
curl --location --request PUT 'http://<EXTERNAL-IP>.nip.io/api/todos/todo/add' --header
'Content-Type: application/json' --data-raw '{
    "title": "Complete DevOps Assignment",
    "description": "Deploy Spring Boot and MySQL multi-tier app to GKE cluster",
    "completed": false
}'
```